Assignment Group

- Each group should have **five** members.
- Please fill in your team information using the link below before **11th April**.
  https://docs.google.com/spreadsheets/d/10-LTTdxGbo8ce_CZ1_hGcnGzdb_3ph9ocKK7CJAYXIA/edit?usp=sharing

- If you do not have a team, please contact our head TA, **Jie Chen** (jie.chen1@ucdconnect.ie) before 11th April.
- The head TA has the right to add a new team member to a team with fewer than five members.
- The penalty for late submission: 10% deduction.
- Failure to fill out the form before 11th April will result in a penalty (10% deduction).

LECTURE 5:

STRUCTURE QUERY LANGUAGE (SQL) 3

COMP2013J: Databases and Information Systems

Dr. Ruihai Dong (ruihai.dong@ucd.ie)
UCD School of Computer Science
Beijing-Dublin International College

Today's Outline

- Limiting the number of rows (LIMIT)
- AUTO_INCREMENT fields.
- Ordering Data (ORDER BY)
- Aggregate Queries (COUNT, MAX, MIN, SUM, AVG)
- Grouping Data (GROUP BY)
- Nested Queries
- Views

LIMITING THE NUMBER OF ROWS

Limiting the Number of Rows

Joe Rodeo Diamond Men's Watch - JUNIOR black 27 ctw

£21,600.00

FREE Delivery
Usually dispatched within 4 to 5 days.

MontBlanc Heritage Automatic Silver Dial Mens Watch 110714

£20,319.51

FREE Delivery
Only 2 left in stock.

Joe Rodeo Diamond Men's Watch - MASTER black 25 ctw

More buying choices
£19,152.00 (1 new offer)

Joe Rodeo Diamond Men's Watch - JUNIOR gold 25.5 ctw

★★★★★ ⌄ 2

More buying choices
£18,168.00 (1 new offer)

Joe Rodeo Diamond Men's Watch - JUNIOR silver 25.5 ctw

More buying choices
£18,168.00 (1 new offer)

Longines Record White Dial with Roman Numeral Markers Steel Men'...

£17,711.00

Usually dispatched within 4 to 5 days.

TAG HEUER CAR5A90.FC6415 Carrera HEUER Calibre 02T TOURBILLON...

More buying choices
£17,702.11 (1 new offer)

Joe Rodeo Diamond Men's Watch - PILOT black 17 ctw

£17,256.00

FREE Delivery

← Previous 1 2 3 ... 7 Next →

LIMIT

employee

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

I have uploaded these tables to Moodle as "**employee.db**" so you can try these queries yourself.

We can use the **LIMIT** keyword to reduce the number of rows returned by a SELECT query.

To show only 4 employees' data (Page 1):

```
SELECT * FROM employee LIMIT 4;
```

The ROW_COUNT specifies the maximum number of rows to return.

LIMIT

employee

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

OFFSET 4

ROW_COUNT 4

To show 5-8 employees' data (Page 2):

```
SELECT * FROM employee LIMIT 4,4;
```

The ROW_COUNT specifies the maximum number of rows to return.

The OFFSET value is used to specify which row to start from retrieving data

LIMIT

employee

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

OFFSET 8

ROW_COUNT 4

To show 9-12 employees' data (Page 3):

```
SELECT * FROM employee LIMIT 8,4;
```

Question

```
SELECT * FROM employee LIMIT 10,1;
```

employee

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

Question

```
SELECT * FROM employee LIMIT 10,1;
```

employee

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

OFFSET 10 →

} ROW_COUNT 1

AUTO INCREMENT

AUTO_INCREMENT

- If you use an INT value as a primary key, you can ask the RDBMS to automatically generate the values for you using **AUTO_INCREMENT**.
- The values begin at 1 and increase by 1 each time you insert a new row.
- Put a NULL value into the AUTO_INCREMENT field when inserting a row.

AUTO_INCREMENT Example

- `CREATE TABLE test (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(30));`
- Two ways to insert:
  1. Don't provide a value for the AUTO_INCREMENT field, e.g. `INSERT INTO test (name) VALUES ('John');`
  2. Insert a NULL value into the AUTO_INCREMENT field, e.g. `INSERT INTO test VALUES( NULL, 'Barry');`

```
mysql> SELECT * FROM test;
+----+-------+
| id | name  |
+----+-------+
|  1 | John  |
|  2 | Barry |
+----+-------+
2 rows in set (0.00 sec)
```

ORDERING DATA

ORDER BY

- The ORDER BY clause, at the end of the query, orders the rows of the result.

- Example:
- `SELECT * FROM employee ORDER BY lastname, firstname;`
  - Employees are ordered by `lastname`.
  - If two employees have the same last name, they will be ordered by their `firstname` attributes.
  - The default behaviour is to sort in **ascending** order (i.e. from A to Z).

ORDER BY

- Example:

- `SELECT * FROM employee ORDER BY salary DESC;`
  - Employees are ordered by their `salary`.
  - Here, we have specified that they are to be ordered in **descending order** (DESC), from the highest salary to the lowest.

AGGREGATE QUERIES

Aggregate queries

- The result of an aggregate query depends on **sets of rows**.

- SQL-2 offers five aggregate operators:
  - `COUNT`
  - `SUM`
  - `MAX`
  - `MIN`
  - `AVG`

COUNT (count values/rows)

- COUNT returns the **number** of rows or distinct values;

- Examples
  - Find the number of employees (number of rows/tuples in the table):

```
SELECT COUNT(*) FROM employee;
```

  - Find the number of values in a particular column (NULL values are not counted):
    - `SELECT COUNT(salary) FROM employee;`

  - Find the number of different values on the attribute salary for all the rows in the employee table:

```
SELECT COUNT(DISTINCT salary) FROM employee;
```

COUNT (count values/rows)

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

```
+----------+
| count(*) |
+----------+
|        3 |
+----------+
```

- Examples
  - Find the number of employees (number of rows/tuples in the table) in department 10:

```
SELECT COUNT(*) FROM employee where deptno=10;
```

COUNT (count values/rows)

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

```
+-----------------------+
| COUNT(DISTINCT deptno) |
+-----------------------+
|                     6 |
+-----------------------+
```

- Examples
  - Find the number of different values on the attribute deptno for all the rows in the employee table:

```
SELECT COUNT(DISTINCT deptno) FROM employee;
```

MAX and MIN

- The MAX function returns the **maximum** value in a set of values.
- The MIN function returns the **minimum** value in a set of values.

- Example
  - Find the highest/lowest salary paid to any employee:

```
SELECT MAX(salary) FROM employee;
SELECT MIN(salary) FROM employee;
```

MAX and MIN

- Example
  - Find the highestsalary paid to any employee:

```
SELECT MAX(salary) FROM employee;
```

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

```
+-------------+
| MAX(salary) |
+-------------+
|       55000 |
+-------------+
```

SUM and AVG

- The SUM function returns the **sum** of a set of value (ignores NULL values).
- The AVG function calculates the **average** value of a set of values (ignores NULL values).

- Example
  - Find the total salary bill for all employees:

```
SELECT SUM(salary) FROM employee;
```

  - Find the average salary of employees:

```
SELECT AVG(salary) FROM employee;
```

Aggregate queries with joins

- Find the highest paid employee who is a teacher.

```
SELECT MAX(salary) FROM employee
  WHERE job='Teacher';
```

- Find the average salary of all employees in the design department

```
SELECT AVG(salary)
  FROM employee
  INNER JOIN department USING(deptno)
  WHERE deptname='Design';
```

Aggregate queries and target list

- Incorrect query:

```
SELECT firstname, familyname, MAX(salary)
  FROM employee;
```

- Attributes **must** have same number of results.

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

Aggregate queries and target list

- Find the maximum and minimum salaries of all employees:

```
SELECT MAX(salary), MIN(salary) FROM employee;
```

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

GROUPING DATA

Grouping data (GROUP BY)

- Queries may apply aggregate operators to **subsets of rows**.

- Find the sum of salaries of all the employees of the same department:

```
SELECT deptno, deptname, SUM(salary) FROM employee
       INNER JOIN department USING(deptno)
       GROUP BY deptno;
```

Grouping Data

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

```
+--------+-----------------+-------------+
| deptno | deptname        | SUM(salary) |
+--------+-----------------+-------------+
|     10 | Training        |      142000 |
|     20 | Design          |      120000 |
|     30 | Implementation  |       74000 |
|     45 | Customer Service|       34000 |
|     50 | Test            |       42000 |
|     60 | Deploy          |       55000 |
+--------+-----------------+-------------+
6 rows in set (0.00 sec)
```

SELECT deptno, deptname, SUM(salary), AVG(salary), MIN(salary), MAX(salary) FROM employee INNER JOIN department USING(deptno) GROUP BY deptno

```
+--------+-----------------+-------------+-------------+-------------+-------------+
| deptno | deptname        | SUM(salary) | AVG(salary) | MIN(salary) | MAX(salary) |
+--------+-----------------+-------------+-------------+-------------+-------------+
|     10 | Training        |      142000 |  47333.3333 |       45000 |       50000 |
|     20 | Design          |      120000 |  40000.0000 |       25000 |       50000 |
|     30 | Implementation  |       74000 |  24666.6667 |       20000 |       29000 |
|     45 | Customer Service|       34000 |  34000.0000 |       34000 |       34000 |
|     50 | Test            |       42000 |  42000.0000 |       42000 |       42000 |
|     60 | Deploy          |       55000 |  55000.0000 |       55000 |       55000 |
+--------+-----------------+-------------+-------------+-------------+-------------+
6 rows in set (0.00 sec)
```

GROUP BY – How it works

- First, the query is executed without GROUP BY and without aggregate operators.
- Then the query result is divided in subsets with the same values for the attributes appearing after the group by clause.
- Finally, the aggregate operator is applied separately to each subset.

GROUP BY – How it works

First, the query is executed without GROUP BY and without aggregate operators.

deptno	empno	firstname	familyname	job	salary	deptname	office	division	managerno
10	1234	Sean	Russell	Teacher	50000	Training	Lansdowne	D1	4567
20	1238	Brendan	Macken	Technician	25000	Design	Belfield	D2	1899
30	1345	Ronan	O'Gara	Manager	29000	Implementation	Donnybrook	D1	1345
20	1555	Sean	O'Brien	Designer	50000	Design	Belfield	D2	1899
30	1585	Denis	Hickie	Architect	20000	Implementation	Donnybrook	D1	1345
20	1899	Brian	O'Driscoll	Manager	45000	Design	Belfield	D2	1899
30	2525	Peter	Stringer	Designer	25000	Implementation	Donnybrook	D1	1345
10	4567	Jamie	Heaslip	Manager	47000	Training	Lansdowne	D1	4567
10	6542	Leo	Cullen	Teacher	45000	Training	Lansdowne	D1	4567
45	6675	Abraham	Macken	Designer	34000	Customer Service	Stillorgan	D14	6675
60	7878	Tommy	Bowe	Manager	55000	Deploy	Ashtown	D15	7878
50	8888	Paul	O'Connell	Driver	42000	Test	Sandyford	D18	8888

SELECT deptno, deptname, SUM(salary), AVG(salary), MIN(salary), MAX(salary) **FROM employee INNER JOIN department USING(deptno)** GROUP BY deptno

GROUP BY – How it works

Then the query result is divided in subsets with the same values for the attributes appearing after the group by clause.

10	1234	Sean	Russell	Teacher	50000	Training	Lansdowne	D1	4567
10	4567	Jamie	Heaslip	Manager	47000	Training	Lansdowne	D1	4567
10	6542	Leo	Cullen	Teacher	45000	Training	Lansdowne	D1	4567

20	1238	Brendan	Macken	Technician	25000	Design	Belfield	D2	1899
20	1555	Sean	O'Brien	Designer	50000	Design	Belfield	D2	1899
20	1899	Brian	O'Driscoll	Manager	45000	Design	Belfield	D2	1899

30	1345	Ronan	O'Gara	Manager	29000	Implementation	Donnybrook	D1	1345
30	1585	Denis	Hickie	Architect	20000	Implementation	Donnybrook	D1	1345
30	2525	Peter	Stringer	Designer	25000	Implementation	Donnybrook	D1	1345

45	6675	Abraham	Macken	Designer	34000	Customer Service	Stillorgan	D14	6675

50	8888	Paul	O'Connell	Driver	42000	Test	Sandyford	D18	8888

60	7878	Tommy	Bowe	Manager	55000	Deploy	Ashtown	D15	7878

SELECT deptno, deptname, SUM(salary), AVG(salary), MIN(salary), MAX(salary) FROM employee INNER JOIN department USING(deptno) **GROUP BY deptno**

GROUP BY – How it works

Finally, the aggregate operator is applied separately to each subset.

10	1234	Sean	Russell	Teacher	50000	Training	Lansdowne	D1	4567
10	4567	Jamie	Heaslip	Manager	47000	Training	Lansdowne	D1	4567
10	6542	Leo	Cullen	Teacher	45000	Training	Lansdowne	D1	4567

20	1238	Brendan	Macken	Technician	25000	Design	Belfield	D2	1899
20	1555	Sean	O'Brien	Designer	50000	Design	Belfield	D2	1899
20	1899	Brian	O'Driscoll	Manager	45000	Design	Belfield	D2	1899

30	1345	Ronan	O'Gara	Manager	29000	Implementation	Donnybrook	D1	1345
30	1585	Denis	Hickie	Architect	20000	Implementation	Donnybrook	D1	1345
30	2525	Peter	Stringer	Designer	25000	Implementation	Donnybrook	D1	1345

45	6675	Abraham	Macken	Designer	34000	Customer Service	Stillorgan	D14	6675

50	8888	Paul	O'Connell	Driver	42000	Test	Sandyford	D18	8888

60	7878	Tommy	Bowe	Manager	55000	Deploy	Ashtown	D15	7878

SELECT **deptno, deptname, SUM(salary), AVG(salary), MIN(salary), MAX(salary)**
FROM employee INNER JOIN department USING(deptno) GROUP BY deptno

Group predicates

- When conditions are on the result of an aggregate operator, it is necessary to use the HAVING clause.

- Find the departments where the average salary is over 40000.

```
SELECT deptno FROM employee GROUP BY deptno
     HAVING AVG(salary) > 40000;
```

WHERE or HAVING?

- Only predicates containing aggregate operators should appear in the argument of the HAVING clause.

- Find the name of the departments where the average salary is over 40000.

```
SELECT deptname FROM employee
  INNER JOIN department USING(deptno)
  GROUP BY deptname HAVING AVG(salary) > 40000;
```

How it works? (Good example :D)

```
SELECT d.deptno, deptname, AVG(salary)
FROM employee as e INNER JOIN department as d ON e.deptno=d.deptno   1
WHERE d.deptno!=10   2
GROUP BY d.deptno   3
HAVING AVG(salary)>30000;   4
```

NESTED QUERIES

Nested Queries

- A **nested query** is an SQL query that is contained within another query.
  - The nested query is sometimes called a **subquery**.
- Most commonly used in the WHERE clause of a query.
- Subqueries can return:
  - A single value (known as a **scalar**)
  - A single column.
  - A single row.
  - A table (multiple columns/rows)

Scalars and single columns are the most common usages of subqueries.

Nested Queries: Scalar Value

- When a subquery returns a single value (scalar), it can be used in the same way as an ordinary single value.

- Find the name of the employee(s) who earn the most money:
  - SELECT firstname, familyname FROM employee WHERE salary=(SELECT MAX(salary) FROM employee);

Nested Queries: Single Columns

- A nested query returning a single column can be thought of as a **set of values**.

- We can compare attributes with values from this set to see if it's equal, greater than, less than, etc.
  - Using = > < >= <= <> !=

- Two other keywords are important:
  - ANY: returns true if the comparison is true for **any** value in the set.
  - ALL: returns true if the comparison is true for **all** the values in the set.

Nested Query: Single Columns Example 1

- Find the names of employees who work in departments in Division 'D1'.

```
SELECT firstname, familyname FROM employee
  WHERE deptno = ANY(SELECT deptno FROM
    department WHERE division='D1');
```

1. The subquery finds a list of all the department numbers (deptno) for departments that are in division D1.
2. When selecting from the 'employee' table, it will match any employee whose 'deptno' is in the set returned by the subquery.

Nested Query: Single Columns Example 1

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

```
SELECT firstname,
familyname FROM employee
  WHERE deptno =
ANY(SELECT deptno FROM
    department WHERE
division='D1');
```

deptno	deptname	office	division	managernumber
10	Training	Lansdown	D1	4567
20	Design	Belfield	D2	1899
30	Implementation	Donnybrook	D1	1345
40	Finance	Ballsbridge	D3	1595

Nested Query: Single Columns Example 2

- Find the employees of department number 10 who have the same first name as a member of department 20.

```
SELECT firstname, familyname FROM employee
  WHERE deptno=10 AND firstname=ANY(
    SELECT firstname FROM employee WHERE
    deptno=20);
```

- (Of course, this particular requirement can also be achieved without a nested query.)

Nested Query: Single Columns Example 2

- Find the employees of department number 10 who have the same first name as a member of department 20.

```
SELECT firstname, familyname FROM employee
  WHERE deptno=10 AND firstname=ANY(
    SELECT firstname FROM employee WHERE
    deptno=20);
```

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

Nested Query: Single Columns Example 3

- Find the name of the Department in which there is no employee named Sean.

```
SELECT deptname FROM department
    WHERE deptno != ALL(SELECT DISTINCT deptno
      FROM employee WHERE firstname='Sean');
```

- Note that we change from ANY to ALL because this is a negative match.
- The subquery finds the ‘deptno’ for departments who do have an employee named ‘Sean’.
- We want to find departments whose numbers are different to all of these.

Nested Query: Single Columns Example 3

empno	firstname	familyname	job	salary	deptno
1234	Sean	Russell	Teacher	50000	10
4567	Jamie	Heaslip	Manager	47000	10
6542	Leo	Cullen	Teacher	45000	10
6675	Abraham	Macken	Designer	34000	45
1238	Brendan	Macken	Technician	25000	20
1555	Sean	O'Brien	Designer	50000	20
1899	Brian	O'Driscoll	Manager	45000	20
2525	Peter	Stringer	Designer	25000	30
1585	Denis	Hickey	Architect	20000	30
1345	Ronan	O'Gara	Manager	29000	30
8888	Paul	O'Connell	Driver	42000	50
7878	Tommy	Bowe	Manager	55000	60

```
SELECT deptname FROM
department
   WHERE deptno !=
ALL(SELECT DISTINCT deptno
     FROM employee WHERE
firstname='Sean');
```

deptno	deptname	office	division	managernumber
10	Training	Lansdown	D1	4567
20	Design	Belfield	D2	1899
30	Implementation	Donnybrook	D1	1345
40	Finance	Ballsbridge	D3	1595

Nested Query: Single Columns Example 4

- Find the name of all employees who earn more money than everybody in the Implementation department.

- ```
  SELECT firstname, familyname FROM employee
    WHERE salary > ALL(SELECT salary FROM
    employee JOIN department USING(deptno)
    WHERE deptname='Implementation');
  ```
  ```

VIEWS

Views

- SQL provides the ability to use **views** in your schema.
- A view is a **virtual table** based on a query.
- It looks just like a normal table, but it is not stored directly in the database.
- We can allow users to see a subset of one or more tables, without giving them access to the table itself.

Views

- Every view has a **name** and a **SELECT query** that defines it.

- Example
- ```
  CREATE VIEW manager AS
     SELECT * FROM employee
     WHERE empno=ANY(SELECT managerno FROM department);
  ```

- After this, ‘managers’ looks just like an ordinary table.
- Changes in the ‘employee’ table will automatically be seen in the ‘manager’ table.
  ```